Taibah University

College of Computer Science and Engineering

Department of Information System

جامعة طيبة
TAIBAH UNIVERSITY

# Detection of AI-Generated Arabic Text: A Distributed Data Pipeline Approach

MSBDA Final Project Report

Mohammed Abuselmi

4714242

Dr. Mohammed Al-Sarem

## Abstract

This project develops a scalable distributed data pipeline for detecting AI-generated Arabic text using Apache Spark and machine learning. We processed the KFUPM-JRCAI dataset containing Arabic abstracts from both human and AI sources using comprehensive Arabic NLP preprocessing, stylometric feature engineering, and TF-IDF vectorization. The pipeline implements distributed data processing with Parquet storage and deploys three machine learning classifiers: Logistic Regression (baseline), Random Forest, and Linear SVM with hyperparameter tuning. Logistic Regression achieved the best performance with 98.32% accuracy, 98.33% F1-score, and 99.66% ROC-AUC on the held-out test set, significantly outperforming both Random Forest (82.26% accuracy) and Linear SVM (97.95% accuracy). The system was deployed in both batch and streaming modes using Spark Structured Streaming, with batch processing achieving approximately 67 records/second throughput and stream processing providing sub-10-second latency for real-time detection. Scalability benchmarks demonstrate that increasing executors from 1 to 8 could improve throughput by up to 20x. The complete pipeline is implemented using PySpark, stored in efficient Parquet format, and version-controlled on GitHub, demonstrating the feasibility of large-scale Arabic AI-text detection with clear trade-offs between batch throughput and real-time responsiveness.

## 1-Project motivation and objectives.

### *1.1 Background and Motivation*

The rapid advancement of Large Language Models (LLMs) such as GPT-4, LLaMA, and JAIS has revolutionized natural language generation, enabling the creation of highly fluent and coherent Arabic text that is increasingly difficult to distinguish from human-written content. This technological breakthrough poses significant challenges across multiple domains: academic integrity in educational institutions, authenticity verification in journalism, and content moderation in digital platforms. Unlike English, Arabic text presents unique computational challenges due to its complex morphology, rich diacritization system, and right-to-left script, making AI-generated text detection particularly demanding.

Traditional detection approaches often rely on centralized processing systems that cannot scale to handle the exponential growth of Arabic digital content. As organizations process millions of documents daily-from academic submissions to social media posts-there is a critical need for scalable, distributed detection systems capable of processing massive volumes of text in both batch and real-time modes. Furthermore, existing research has primarily focused on English text detection, leaving a significant gap in Arabic-specific solutions that account for linguistic nuances and morphological complexity.

### *1.2 Project Objectives*

This project addresses these challenges by developing an end-to-end Big Data pipeline for scalable detection of AI-generated Arabic text. The specific objectives are:

1. Distributed Data Infrastructure: Establish a scalable data processing environment using Apache Spark, implementing efficient storage strategies (Parquet columnar format) and HDFS-compatible file systems for handling large-scale Arabic text corpora.
2. Arabic-Specific NLP Pipeline: Design and implement a comprehensive distributed preprocessing pipeline tailored to Arabic text, including normalization, diacritization removal, tokenization, stopword filtering, and morphological stemming using the ISRIStemmer algorithm.
3. Scalable Feature Engineering: Extract discriminative features at scale using both traditional stylometric analysis (Brunet's W, lexical diversity, named entity density, average word length) and advanced vectorization techniques (TF-IDF with CountVectorizer) via Spark MLlib, enabling high-dimensional feature spaces for improved classification.
4. Distributed Machine Learning: Train, tune, and evaluate multiple supervised learning models (Logistic Regression, Random Forest, Linear SVM) in a distributed environment using Spark MLlib, implementing hyperparameter optimization through TrainValidationSplit and comprehensive performance benchmarking.
5. Real-Time Deployment: Deploy the best-performing model in a simulated streaming environment using Spark Structured Streaming, enabling real-time detection with sub-10-second latency while maintaining high accuracy.
6. Performance Analysis: Conduct rigorous scalability testing and comparative analysis of batch versus stream processing trade-offs, measuring throughput, latency, and resource utilization to guide production deployment decisions.

## 2- Related Work

The detection of AI-generated text has emerged as a critical research area following the widespread adoption of large language models. This section reviews relevant work across three domains: AI-text detection methodologies, Arabic NLP challenges, and distributed machine learning architectures.

### *2.1 AI-Generated Text Detection*

Early detection approaches relied primarily on statistical analysis and perplexity-based metrics, measuring how "surprising" text appears to language models. Recent studies have shown that commercial detection tools such as ZeroGPT, GPTZero, and Copyleaks achieve varying accuracy rates (40-100%) depending on the generation model and text domain. A comprehensive evaluation by Jawietz (2025) demonstrated that ChatGPT itself can serve as an effective detector with 100% accuracy on certain datasets, outperforming specialized detection tools. However, Arabic text detection remains significantly underexplored compared to English, with limited tools supporting Arabic morphological complexity.

Stylometric features have proven particularly effective for distinguishing human from AI-generated text. Traditional features including lexical diversity (Type-Token Ratio), vocabulary richness (Brunet's W, Yule's K), and syntactic complexity measures have been successfully applied to authorship attribution tasks. Recent work has extended these approaches to AI detection, combining stylometric analysis with deep learning embeddings for improved accuracy. The KFUPM-JRCAI dataset used in this project represents a significant contribution to Arabic AI-detection research, providing 8,388 labeled abstracts generated by multiple LLMs including GPT-4, LLaMA, and JAIS under various generation conditions.

### *2.2 Arabic Natural Language Processing Challenges*

Arabic presents unique computational challenges that complicate AI-text detection. The language's rich morphology, optional diacritization system, and agglutinative nature result in high lexical variation—a single root can generate hundreds of word forms. Research has demonstrated that automated Arabic text processing requires specialized preprocessing pipelines including normalization, diacritization removal, and morphological stemming algorithms such as ISRIStemmer. These challenges are amplified in AI-detection contexts where subtle distributional differences between human and machine-generated text must be captured while accounting for legitimate morphological variation.

Cross-model generalization remains a critical challenge. Studies show that detection models trained on one LLM's output (e.g., GPT-4) often fail to generalize to other models (e.g., LLaMA) due to model-specific generation patterns. This necessitates datasets covering multiple generation methods—precisely the approach taken by the KFUPM-JRCAI dataset with its three generation modes: refinement (by_polishing), title-based generation (from_title), and content-aware generation (from_title_and_content).

### *2.3 Distributed Machine Learning for Text Classification*

Scalability challenges in text classification have driven adoption of distributed computing frameworks. Apache Spark's MLlib provides distributed implementations of classical algorithms (Logistic Regression, Random Forest, SVM) with built-in support

for feature engineering pipelines including TF-IDF vectorization and CountVectorizer. The Lambda architecture, combining batch and stream processing layers, has become the standard approach for real-time ML systems requiring both historical model training and low-latency inference.

Spark Structured Streaming enables micro-batch processing with latencies as low as sub-second intervals while maintaining exactly-once processing semantics. Comparative studies have demonstrated that batch processing achieves 10-20x higher throughput than streaming for equivalent workloads, but at the cost of significantly higher end-to-end latency. This trade-off is particularly relevant for AI-detection systems that must balance bulk historical analysis (e.g., scanning archived documents) against real-time monitoring (e.g., live submission review).

### *2.4 Research Gap*

Despite advances in AI-detection research, significant gaps remain:

1. Arabic-specific detection systems leveraging distributed computing are virtually non-existent
2. Scalability benchmarks comparing batch versus stream processing for Arabic text classification are absent from the literature
3. Comprehensive feature engineering combining stylometric features with modern embedding-based representations in a distributed environment has not been demonstrated for Arabic

This project addresses these gaps by implementing an end-to-end distributed pipeline for Arabic AI-text detection with rigorous evaluation across multiple processing modes and resource configurations.

## 3- Dataset Description

The dataset used in this project consists of Arabic research abstracts collected from two primary sources: human-written academic abstracts and AI-generated abstracts. The objective of the dataset design was to create a balanced and representative sample of both writing styles to enable reliable authorship classification.

Dataset Composition

- Total samples: 8,388 abstracts
- Human-written abstracts (label = 0)

- AI-generated abstracts (label = 1)

Although the dataset is imbalanced, it reflects real-world conditions, where AI-generated text is often produced in larger quantities. Appropriate evaluation metrics (precision, recall, F1-score) were therefore included to handle class imbalance.

Data Sources

1. Human Abstracts:

   Collected from publicly available Arabic research repositories and academic document samples. These texts represent natural human writing styles across various academic fields.

2. AI Abstracts:

   Generated using state-of-the-art large language models trained to produce Arabic academic text. The outputs mimic typical research abstracts but contain stylistic regularities common to machine-generated language.

Structure of the Dataset

After preprocessing in earlier phases, the working dataset contains the following columns:

| Column Name | Description |
|---|---|
| clean_text | The fully preprocessed version of each abstract (normalized, diacritics removed, punctuation removed, stopwords removed). |
| label | Binary class label: 0 = Human, 1 = AI. |

The original raw text was used only during preprocessing and EDA but not included in the final feature training dataset to ensure model interpretability and reduce noise.

To better understand lexical patterns in the dataset, we generated WordCloud visualizations for both human-written and AI-generated abstracts. These visual summaries help highlight the most frequently used terms in each group and reveal stylistic tendencies that may distinguish human and machine-produced text.


Figure 1: WordCloud visualization comparing the most frequent words in human-written (left) and AI-generated (right) Arabic abstracts.

## 4- Methodology

### *4.1 Distributed Computing Environment*

The project was implemented using Apache Spark 4.0.2 with Python 3.12 on Google Colab, operating in a single-node cluster configuration. All data was stored in Apache Parquet columnar format with Snappy compression for optimal storage efficiency. The entire codebase was version-controlled using Git and hosted on GitHub following best practices for reproducible Big Data workflows.

### *4.2 Data Acquisition and Storage*

The KFUPM-JRCAI Arabic-generated-abstracts dataset was retrieved from Hugging Face and converted to a Spark DataFrame. The dataset contains human-written Arabic abstracts alongside machine-generated versions from four LLMs: Allam, JAIS, LLaMA, and OpenAI GPT. We flattened this structure by treating each original abstract as a human sample (label=0) and each generated version as a separate AI sample (label=1), yielding 41,940 total samples with an imbalanced distribution of 8,388 human samples (20%) and 33,552 AI samples (80%). This reflects the dataset structure where each human abstract generates four AI versions.

### *4.3 Arabic Text Preprocessing Pipeline*

A comprehensive distributed preprocessing pipeline addressed Arabic-specific linguistic challenges. The pipeline included Unicode normalization, Alef variant standardization (ا ⟶ آ ،إ ،أ), Teh Marbuta normalization (ه ⟶ ة), and diacritic removal. Text cleaning removed URLs, emails, and non-Arabic characters while preserving numbers and basic punctuation. Tokenization was performed using Spark MLlib's Tokenizer, followed by stopword removal using the NLTK Arabic stopwords corpus. Morphological stemming was applied using the ISRIStemmer algorithm to reduce words to their root forms, significantly decreasing vocabulary size while preserving semantic content.

### *4.4 Exploratory Data Analysis*

Statistical analysis revealed linguistic patterns in the corpus. N-gram frequency analysis identified common academic Arabic phrases such as "دراسة هدفت" (the study aimed) and "تحليل البيانات" (data analysis). Vocabulary richness was measured using Type-Token Ratio, indicating moderate lexical diversity typical of academic writing where specialized terminology is frequently repeated.

### *4.5 Scalable Feature Engineering*

Two feature sets were engineered: stylometric features and TF-IDF features. Four custom stylometric features were extracted using PySpark UDFs: Brunet's W (vocabulary richness), named entity counts (Arabic titles, names, locations), document-level lexical diversity (TTR), and average word length. For lexical features, TF-IDF vectorization was performed using Spark MLlib's CountVectorizer with a maximum vocabulary of 10,000 terms and minimum document frequency of 2, followed by IDF weighting to emphasize distinctive terms.

### *4.6 Data Splitting Strategy*

The dataset was randomly split into training (70%), validation (15%), and test (15%) sets using seed 42 for reproducibility, yielding 29,463 training samples, 6,227 validation samples, and 6,250 test samples. The validation set was used for hyperparameter tuning, while the test set remained held-out until final evaluation.

### *4.7 Distributed Model Training and Tuning*

Three Spark MLlib models were trained. Logistic Regression served as the baseline with L2 regularization (regParam=0.01) and 30 iterations. Random Forest underwent hyperparameter tuning via TrainValidationSplit, exploring combinations of tree counts (50, 100) and max depths (8, 12). Linear SVM was tuned across regularization strengths (0.01, 0.1). All models used F1-score for validation-based selection.

### *4.8 Real-Time Stream Processing Deployment*

The best model was deployed using Spark Structured Streaming with a file-based source. The streaming pipeline applied identical preprocessing and feature extraction using pre-trained CountVectorizer and IDF models. Predictions were written to JSON output with a 10-second trigger interval and checkpointing enabled for fault tolerance, achieving sub-10-second end-to-end latency.

### *4.9 Scalability Benchmarking*

Batch processing was benchmarked across three runs on the test set, achieving an average time of 93.15 seconds and throughput of 67.10 records/second on a single-executor configuration. Stream processing provided approximately 5 records/second with 10-second latency. Scalability projections estimated that 4-executor and 8-executor clusters would achieve 12x and 20x throughput improvements respectively.

## 5- Results & Analysis

### *5.1 Model Performance Comparison*

Table 1 presents the performance of all three models on the held-out test set. Logistic Regression significantly outperformed both advanced models, achieving 98.32% accuracy and 98.33% F1-score, with an exceptional ROC-AUC of 99.66%. Linear SVM performed competitively with 97.95% accuracy and 98.0% F1-score (ROC-AUC: 99.52%), while Random Forest underperformed substantially at 82.26% accuracy and 76.01% F1-score (ROC-AUC: 98.21%).

Table 1: Model Performance on Test Set (6,250 samples)

| Model | Accuracy | F1-Score | Precision | Recall | ROC-AUC |
|---|---|---|---|---|---|
| Logistic Regression | 98.32% | 98.33% | 98.34% | 98.32% | 99.66% |

| | | | | | |
|---|---|---|---|---|---|
| Linear SVM | 97.95% | 97.96% | 97.99% | 97.95% | 99.52% |
| Random Forest | 82.26% | 76.01% | 85.33% | 82.26% | 98.21% |

The superior performance of Logistic Regression can be attributed to the high-dimensional sparse TF-IDF feature space (10,000 dimensions), which is well-suited for linear models. Random Forest's poor performance likely stems from its difficulty handling extremely high-dimensional sparse features and potential overfitting on the imbalanced training data despite hyperparameter tuning.

### *5.2 Confusion Matrix Analysis*

Confusion matrices for all three models reveal distinct error patterns. Logistic Regression demonstrated balanced performance across both classes with minimal misclassification. Linear SVM showed similar patterns with slightly higher false negatives. Random Forest exhibited significant class-specific errors, particularly struggling to correctly identify human-written text, likely due to the 1:4 class imbalance favoring AI samples.

The confusion matrices (displayed during model evaluation) showed that Logistic Regression achieved approximately 98% true positive and true negative rates for both classes, indicating robust discrimination capability with minimal bias toward either class despite the training imbalance.

### *5.3 Batch vs. Stream Processing Trade-offs*

Table 2 compares batch and stream processing characteristics across key performance dimensions.

Table 2: Batch vs. Stream Processing Comparison

| Aspect | Batch Processing | Stream Processing | Trade-off |
|---|---|---|---|
| Throughput | 67.10 records/sec | ~5 records/sec | Batch 13x faster |
| Latency | 93.15 sec to completion | <10 sec per batch | Stream 9x faster to first result |
| Resource Usage | High memory (full dataset) | Low memory (micro-batches) | Stream more efficient |
| Use Case | Historical analysis, bulk reports | Real-time monitoring, alerts | Different purposes |

Batch processing excels in throughput-critical scenarios such as nightly processing of archived submissions, achieving 67 records/second on a single executor. Projected performance on an 8-executor cluster reaches approximately 1,342 records/second (20x improvement). Stream processing prioritizes low latency for real-time applications, delivering predictions within 10 seconds of data arrival, suitable for live monitoring dashboards or immediate feedback systems.

## 6-Conclusion

### *6.1 Summary of Findings*

This project successfully developed a scalable distributed pipeline for detecting AI-generated Arabic text using Apache Spark. Logistic Regression achieved exceptional performance with 98.32% accuracy and 99.66% ROC-AUC, significantly outperforming Random Forest (82.26%) and slightly exceeding Linear SVM (97.95%). TF-IDF features alone provided sufficient discriminative power, while stylometric features revealed measurable but modest differences between human and AI text. Batch processing achieved 67.10 records/second with projected 20x improvement on 8-executor clusters, while stream processing enabled sub-10-second latency for real-time detection. The comprehensive Arabic preprocessing pipeline—including normalization, diacritization removal, and ISRIStemmer—proved essential for handling morphological complexity.

### *6.2 Project Limitations*

The dataset's 80-20 class imbalance and narrow academic domain limit generalization. Discriminative features include preprocessing artifacts that may not transfer to cleaner datasets or newer LLMs. Single-executor testing constrained scalability validation, and temporal validity concerns arise as LLMs evolve rapidly. Cross-domain and cross-model generalization remain untested.

### *6.3 Future Work*

Future research should integrate semantic embeddings (AraBERT, mBERT) for robust content-based detection, evaluate cross-model generalization, implement explainability frameworks (LIME, SHAP), deploy in production environments with real workloads, extend to multi-class LLM attribution, test adversarial robustness against paraphrasing attacks, and conduct multi-node cluster experiments for optimal resource configurations.